TutorAI: An Agentic, Validation-Gated Pipeline for Self-Explaining, Highlight-Synchronized Visual Lessons on an Offline-First Mobile App

Subrath S
Dept. of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bangalore, India
subrath0626@gmail.com

Prasad A M
Dept. of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bangalore, India
prasad-cs@dayanandasagar.edu

***Abstract*—A learner who wants a quick visual answer to "how does this work" usually has to stitch together text, a static picture, and a separate video. We present TutorAI, a system that turns a single typed topic into a short lesson that explains itself. The lesson is a vector diagram whose parts have stable names, a narration script split into ordered segments, and one spoken audio clip per segment. On the phone, each segment lights up the diagram parts it talks about while its audio plays, so the learner always sees exactly what is being described. The hard part of this idea is reliability: a large language model (LLM) will happily write narration that points at a diagram part that does not exist, which breaks the highlighting. We solve this with an agentic generation pipeline built on LangGraph that separates planning, drawing, critique-and-refine of the drawing, narration, and a final deterministic check. A non-LLM validator acts as a hard gate: a lesson is never cached unless every referenced part really exists in the diagram. The backend is a stateless FastAPI service that uses a shared cache keyed by the topic, so a popular topic is generated once and then served instantly and cheaply. The client is an Android application built with Clean Architecture and MVVM that downloads a lesson once and then replays it fully offline, with on-device history. We describe the architecture, the generation pipeline, the reliability mechanisms, and the engineering practices (provider adapters, hermetic testing, containerized deployment, and continuous integration) that make the system reproducible. We also report what the running system produces and discuss the trade-offs we made.**

***Index Terms*—generative AI, large language models, LLM agents, intelligent tutoring, multimedia learning, SVG generation, text-to-speech, FastAPI, Android, offline-first, software architecture**

I. Introduction

Good explanations are often visual and spoken at the same time. A teacher points at a part of a drawing and says what it does. The pointing and the talking happen together, so the listener never has to guess which part the words are about. Most digital learning tools lose this link. A page of text sits next to a fixed picture, and a video lives somewhere else. The learner carries the load of connecting them.

This paper describes TutorAI, a working system that rebuilds this "point and explain" experience automatically. The learner types one topic, for example *Photosynthesis*. The system returns a single, self-contained lesson made of three things: (1) a diagram drawn as Scalable Vector Graphics (SVG) where every meaningful part has a stable, human-readable name; (2) a narration script broken into ordered *segments*, where each segment lists the diagram part names it talks about; and (3) one short spoken audio clip for each segment, with its exact duration measured on the server. On the device the lesson plays segment by segment. For each segment the app highlights the named diagram parts and plays the matching audio. Because the highlight is driven by which clip is playing, the picture and the voice stay in step without any word-level timing and without a network connection.

The central engineering challenge is not getting a model to produce an attractive picture or a fluent script. Modern models do both. The challenge is keeping the two *consistent* with each other. If the narration says "the chloroplast absorbs the light" but the diagram has no part named `chloroplast`, the highlight has nothing to point at and the core promise of the product is broken. Language models are known to produce confident but wrong output, a problem widely studied under the name hallucination [1]. A single prompt that asks for a picture and a matching script in one shot gives us no place to catch this mismatch.

Our answer is to treat generation as a small pipeline of cooperating steps rather than one big request. We build the pipeline with LangGraph [2], a library for expressing an agent as a state graph with conditional edges and loops. The steps are: plan the concepts, draw the SVG, critique and refine the drawing, write the narration against the finished drawing, and finally validate. Validation is done by ordinary code, not by a model. It parses the SVG, collects the real part names, and checks that every name referenced by the narration exists. If a check fails, the pipeline enters a bounded repair loop and tries again with a targeted fix prompt. Only a lesson that passes every check is stored. This deterministic gate is what turns a probabilistic model into a dependable feature.

We make four contributions.

- A concrete system design that produces *self-explaining* visual lessons: an SVG with stable identifiers, segment-level narration tied to those identifiers, and per-segment audio with measured durations.
- An agentic generation pipeline with a separate drawing critique-and-refine loop and a separate narration repair loop, both bounded, and a non-LLM validator that serves as a hard caching gate.
- A pragmatic, cost-aware backend design: a stateless service with a shared topic-keyed cache and request coalescing, behind swappable provider adapters so the choice of LLM and speech engine is configuration, not code.
- An offline-first Android client that achieves deterministic picture-to-voice synchronization with no streaming timing, plus the engineering scaffolding (hermetic tests, containerization, and continuous integration and delivery) that makes the whole system reproducible.

The rest of the paper is organized as follows. Section II surveys the research and engineering background. Section III gives the system overview and the main design decisions. Sections IV and V describe the backend and the client. Section VI covers the patterns, testing, and delivery that support reproducibility. Section VII reports what the system produces. Section VIII discusses limitations, and Section IX concludes.

II. Related Work

Our system sits at the meeting point of several fields: tutoring systems, the psychology of learning from words and pictures, language models and the agents built on top of them, automatic diagram generation, speech synthesis, and offline-first mobile design. We review each in turn and say where TutorAI fits.

A. Intelligent Tutoring Systems and AI in Education

The dream of a patient, one-to-one tutor for every learner is old. Bloom's much-cited "two sigma" study reported that students taught one-to-one performed far better than students in a normal classroom, and framed the open question of how to reach that gain at scale [3]. Intelligent tutoring systems (ITS) were one answer. Cognitive tutors modeled the steps a student takes to solve a problem and gave feedback at each step [4]. A broad review by VanLehn compared human tutors, step-based ITS, and simpler systems, and found that well-built step-based systems can approach the effectiveness of human tutors [5]. Classic ITS, however, were expensive to author: experts hand-built the domain model and the feedback rules for each topic. TutorAI takes a different route. It does not model the learner's steps. It generates a fresh explanatory artifact for any typed topic on demand, trading deep per-topic modeling for broad, instant coverage. The recent arrival of capable language models has reopened this whole space, and several surveys discuss both the promise and the risks of using such models in education [6].

B. Learning from Words and Pictures

Why combine a diagram with narration at all? Decades of work in educational psychology give a clear reason. Paivio's dual coding theory argues that the mind has separate channels for verbal and visual information, and that using both can help memory [7]. Sweller's cognitive load theory warns that working memory is small, so a lesson should avoid wasting it on needless effort [8]. Mayer's cognitive theory of multimedia learning brings these ideas together and yields practical design rules: present matching words and pictures close together in time and space, because making the learner hunt for the link wastes effort. This is the well-known spatial and temporal contiguity principle [9], [10]. TutorAI is a direct application of contiguity. By lighting up the exact diagram parts while the matching sentence is spoken, it removes the search step that a separate caption or a wall of text would force on the learner. Our choice of segment-level synchronization, rather than word-level, is a deliberate balance: it gives strong temporal contiguity while keeping the system deterministic and offline-capable.

C. Large Language Models for Content Generation

The engine behind on-demand generation is the modern language model. The transformer architecture made it practical to train very large models on text [11], and scaling those models produced systems that can follow instructions and perform many tasks from a few examples or a plain description [12]. Later multimodal models, such as the Gemini family used in our backend, extended this to images, audio, and structured output [13]. These models are strong at producing fluent text and, increasingly, at producing structured artifacts like code and markup. But two weaknesses matter for us. First, they hallucinate: they can produce output that looks right but is not faithful to the input or to itself [1]. Second, plain free-text output is hard to consume in software. We lean on *structured output*, where the model is constrained to return data that fits a schema, and we validate that data before trusting it. This combination of generation plus external checking is a recurring theme in reliable LLM systems.

D. LLM Agents, Orchestration, and Self-Correction

A single prompt is a blunt tool for a multi-part task. A growing body of work shows that breaking a task into steps and letting the model reason between them improves results. Chain-of-thought prompting elicits intermediate reasoning and improves performance on harder problems [14]. ReAct interleaves reasoning with actions so a model can use tools and react to what it observes [15]. Two ideas are especially relevant to our design. Self-Refine has a model critique its own output and improve it over several passes [16], and Reflexion lets an agent reflect on feedback and retry [17]. Most directly related to our repair step, self-debugging teaches a model to read error messages and fix its own code [18]; our repair loop has the same shape, except the error messages come from a deterministic validator rather than a runtime. TutorAI uses these ideas in a focused, bounded way: a critique-and-refine

loop improves the diagram before narration is written, and a repair loop fixes the narration when the validator rejects it. To express these loops cleanly we use LangGraph, which models an agent as a state graph with nodes, conditional edges, and cycles, built on the LangChain ecosystem [2], [19]. Unlike open-ended autonomous agents, our graph is small, has a fixed set of nodes, and caps every loop, which keeps cost and latency predictable. The wider practice of programming with these models, including prompt design and tool use, is surveyed by Liu et al. [20].

E. Automatic Diagram and Visualization Generation

Turning a description into a picture has two broad styles. One style produces pixels with image models such as diffusion models [21]. Pixels look rich but are opaque: there is no clean way to name a region or to attach behavior to it. The other style produces a structured drawing, such as SVG, where each shape is an addressable element. SVG is a W3C standard vector format in which any element can carry an `id` attribute [22]. We chose the structured route precisely because we need to address parts by name to drive highlighting, and because vector output scales cleanly on any screen. Generating vector graphics directly is an active area: DeepSVG learns a generative model over the paths and commands that make up an SVG [23], and more recent work produces SVG code from images or text [24]. We do not train such a model; we prompt a general code-capable language model to emit SVG, which is now common [25] but whose quality and self-consistency vary. This variability is exactly why we add an explicit critique-and-refine loop and a deterministic structural check around the model rather than trusting a single drawing pass.

F. Speech Synthesis and Narrated Content

The spoken half of a lesson depends on text-to-speech (TTS). Neural TTS made synthetic voices natural enough for comfortable listening. WaveNet showed that a neural network could generate raw audio of high quality [26], and Tacotron 2 produced speech close to human recordings from text [27]. Modern hosted models, including the Gemini speech model we call through an adapter, expose this as a simple service that returns audio for a piece of text. A detail that matters for synchronization is *duration*. Because we render one clip per segment on the server and measure its exact length from the returned audio bytes, the client can drive the highlight purely from clip boundaries. We never need server-side word timing or client-side streaming alignment, which are the usual sources of drift.

G. Mobile and Offline-First Learning Applications

Finally, a tool meant for studying must work when the network does not. The offline-first idea treats the local device as the source of truth and the network as an enhancement, so the app stays usable on a train, in a basement, or on a weak connection [28]. On Android, the recommended way to build such an app is a layered architecture with a clear separation between the user interface, the domain logic, and the data sources, which the official guidance frames as MVVM over a repository layer [29]. This lines up with Clean Architecture, which pushes dependencies inward toward policy and away from frameworks and details [30]. TutorAI follows this guidance: once a lesson is downloaded, its diagram and audio live in app-private files and its metadata lives in a local database, so replay touches no server.

H. Positioning

In short, TutorAI is not a new model and not a new learning theory. It is a *systems* contribution. It combines an agentic generation pipeline, a deterministic consistency gate, a cost-aware caching backend, and an offline-first client into one working product, and it shows how to make a probabilistic generator reliable enough to drive a feature that depends on exact part-to-word correspondence.

III. System Overview

A. Requirements

The product has a small set of firm requirements. It must accept a free-text topic. It must generate an SVG with stable, semantic part names. It must generate ordered narration segments, each referencing the part names it describes. It must produce one audio clip per segment with a measured duration. It must assemble these into one self-contained lesson. It must serve a cached lesson for a repeated topic. On the device, it must download and persist a lesson, play it with synchronized highlighting, offer play, pause, resume, seek-by-segment, and replay, keep a history of saved lessons, and replay any of them with no network. It must also fail clearly and allow the user to try again.

The non-functional goals follow from the use case. A cache hit should feel instant. A saved lesson must play with zero network calls. The generator must validate structure before caching. Identical topics must be generated once and shared to control cost. The choice of model and speech engine must be swappable. The core logic must be unit-testable. Personal history must never leave the device, since there are no accounts.

B. Architecture

Figure 1 shows the shape of the system. It is a relatively thin client talking to a stateless server, which in turn calls hosted AI services. The client is an Android app. The server is a FastAPI service. The AI services are a Gemini text model and a Gemini speech model, each reached through an adapter so they can be replaced.

C. Key Decisions

Three decisions shape everything else.

Synchronization by pre-rendered timed segments. Rather than align voice to highlight at play time, the server splits the narration into segments, renders one audio clip per segment, and measures each clip's duration. The device simply plays clips in order and highlights the parts named by the current

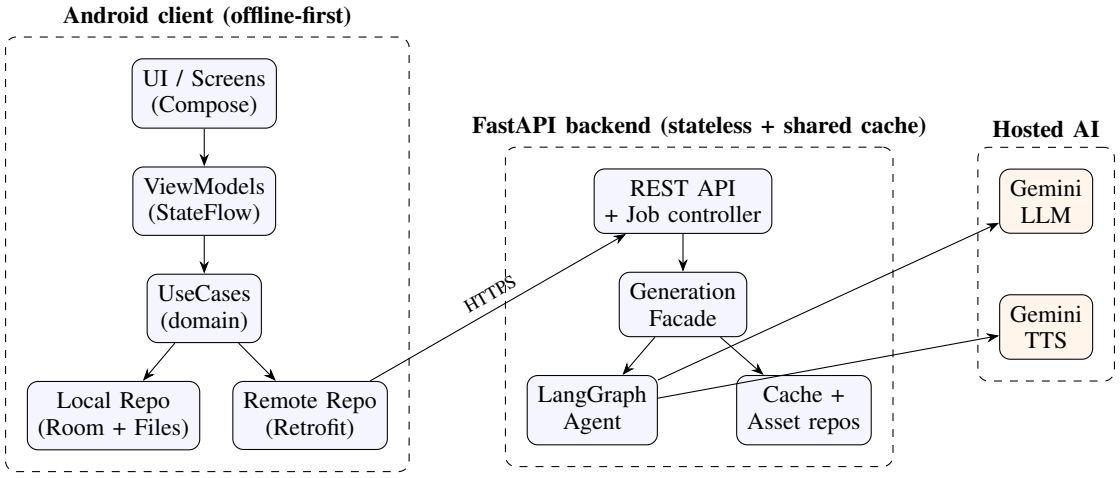


Fig. 1. System architecture. A thin offline-first Android client talks to a stateless FastAPI backend over HTTPS. The backend orchestrates hosted AI through swappable provider adapters and serves repeated topics from a shared cache.

clip. This makes synchronization deterministic and removes any need for the network during playback.

Stateless backend with a shared cache. There are no user accounts on the server. Lessons are cached by a normalized topic key, so a popular topic is generated once and then served to everyone. This is cheaper and faster, and it keeps the server simple to scale and reason about.

Provider adapters. The model and the speech engine sit behind narrow interfaces. The pipeline talks to those interfaces, never to a vendor SDK directly. Choosing a model, or replacing the whole vendor, is a configuration change.

IV. Backend Implementation

The backend is a Python FastAPI application of roughly two thousand lines, organized by responsibility: an API layer, a domain layer of typed entities and schemas, provider adapters, the agent graph, services, and repositories. A single composition root wires these together, which means swapping an implementation happens in one place plus the typed settings object.

A. Configuration over Hard-Coding

Everything that varies between environments is read from environment variables through one typed settings object, following the twelve-factor configuration guideline [31]. This includes the choice of LLM and TTS provider, the generation engine, the model identifiers, the storage backend, and the tuning knobs for the loops. Two points are worth stressing. First, model identifiers are treated as configuration, not code, because hosted model names change over time; a small helper lists the live models so the pinned name can be confirmed. At the time of writing the backend is pinned to a Flash-class Gemini text model (for example `gemini-2.5-flash`) and a Flash-class Gemini speech model, both set in settings rather than in code. Second, the same code base runs on cheap mock providers or on real Gemini providers depending only on settings, which is what keeps tests fast and key-free.

B. Provider Adapters

Two interfaces define what the pipeline needs: an LLM provider that can plan concepts, draw an SVG, critique and refine an SVG, write narration as structured data, and repair a drawing-plus-narration given a list of errors; and a TTS provider that turns text into an audio clip and reports its duration. There are two implementations of each. The mock implementations are deterministic and need no network, which makes them ideal for tests and for local development. The Gemini implementations call the hosted models. The TTS adapter receives raw PCM audio, wraps it as a WAV file, and computes the exact duration from the byte count and sample rate, so the duration is a measured fact rather than an estimate. The adapters also retry transient errors with exponential backoff. A factory builds a matching set of providers from the settings, following the abstract factory pattern.

C. The Generation Pipeline

The heart of the backend is the LangGraph state graph. A single typed state object flows through the nodes. Figure 2 shows the graph. The nodes are as follows.

Planner. The model outlines the concepts and a visual plan for the topic. Planning once, up front, keeps the drawing and the narration aligned to the same set of concepts.

SVG generator. Given the plan, the model draws the SVG and assigns a semantic `id` to every meaningful part, for example `sun`, `leaf`, and `chloroplast`.

Critique and refine. A critique step scores the drawing and lists concrete problems. If the score is below a threshold and budget remains, a refine step redraws with that feedback, then the critique runs again. This loop is capped by a configurable number of passes. If the budget runs out, the best drawing so far is accepted and the pipeline moves on. This is a focused use of the self-critique idea from the literature, applied only to the part that benefits most.

Narration generator. Given the finished drawing, the model writes ordered narration as structured data: each segment has

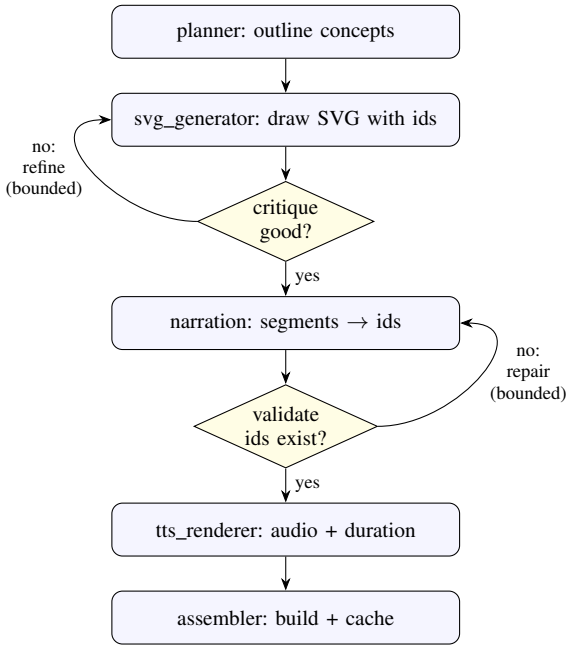


Fig. 2. The generation graph. A bounded critique-and-refine loop improves the drawing before narration is written; a bounded repair loop fixes narration that fails the deterministic validator. A lesson reaches the assembler only after it passes validation.

text and a list of part names. By writing narration *against* the real drawing, the model is steered toward referencing parts that exist.

Validator. This node uses no model. It parses the SVG as XML, confirms it is well-formed and has a `viewBox`, collects the real part names, and checks that every part name referenced by every segment exists. It also rejects empty narration and empty segment text. The result is a simple pass-or-fail with a list of errors.

Repair. If validation fails and retries remain, the repair step is given the exact list of offending names and parse errors and is asked for the minimal correction, then validation runs again. The loop is bounded; if it is exhausted, the job fails cleanly rather than caching a broken lesson.

Renderer. For each segment, the TTS provider synthesizes audio, the clip is stored as an asset, and its measured duration is recorded.

Assembler. A builder assembles the lesson incrementally: the title, the SVG and its stored asset, and each segment with its text, part names, audio asset, and duration. The finished lesson is then cached.

D. The Deterministic Gate

The validator deserves emphasis because it is the reason the feature is trustworthy. A model is probabilistic, but the invariant we depend on is binary: either every referenced part exists or it does not. By enforcing that invariant in plain, testable code and refusing to cache anything that fails it, we convert an unreliable generator into a dependable source of lessons. The repair loop is the bridge: it gives the model a few bounded chances to satisfy the invariant before we give up. This is the practical answer to the hallucination risk noted in the related work.

E. Caching, Jobs, and Coalescing

Because a cold generation can take several seconds, the generate endpoint is asynchronous. A topic is normalized into a key. On a cache hit the manifest is returned at once. On a miss a job is created and the client polls a status endpoint that reports a stage and a percentage until the job reaches a terminal state. Jobs are keyed by the topic key, so two devices asking for the same topic at the same time share one job and one generation. The error responses share one envelope with a code, a message, a retryable flag, and a request identifier, so the client can react consistently. The result that the client downloads is a *manifest*, the wire and storage contract for a lesson. Listing 1 shows its shape.

Listing 1. The lesson manifest (abridged). Audio is always a downloadable asset; the SVG may be inlined or referenced.

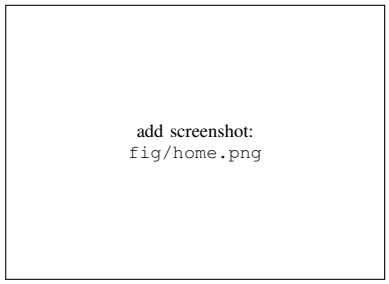
```
{
  "id": "les_8f3a...",
  "topic": "Photosynthesis",
  "title": "How Photosynthesis Works",
  "language": "en", "voice": "Kore",
  "total_duration_ms": 41200,
  "svg": { "asset_id": "ast_svg_01",
           "url": ".../assets/ast_svg_01" },
  "segments": [
    { "index": 0,
      "text": "Sunlight strikes the leaf...",
      "svg_element_ids": ["sun", "leaf"],
      "audio": { "asset_id": "ast_aud_00",
                 "url": ".../assets/ast_aud_00" },
      "duration_ms": 4200 }
  ]
}
```

F. Persistence

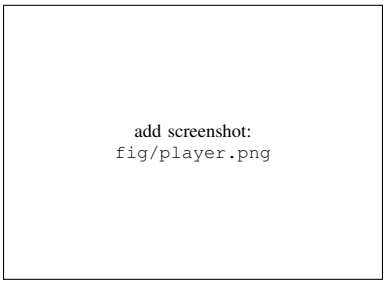
The storage layer is also behind interfaces. A lesson repository hides the lesson cache and an asset repository hides the object store. In development both can run in memory or on the local filesystem; the same interfaces allow a promotion to a database and a cloud object store without touching call sites. This keeps the dev profile light while leaving a clear path to a production profile.

V. Client Implementation

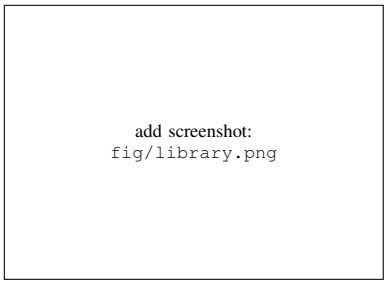
The Android client is a Kotlin application of roughly four thousand lines, built with Jetpack Compose for the user interface and organized with Clean Architecture and MVVM. The presentation layer holds the screens and view models. The domain layer holds pure entities, use cases, and repository interfaces with no Android dependencies. The data layer implements those interfaces with a Retrofit client for the network, a Room database for metadata, and a file store for binary assets. Dependencies point inward, so the domain does not know about the framework. We wire dependencies with a small manual container rather than a framework, a pragmatic choice that traded a little boilerplate for build simplicity. Figure 3 shows the running app: topic entry, the synchronized player, and the offline library.



(a) Topic entry on the home screen



(b) Player: segment 1 of 6, diagram highlighted



(c) Offline library of saved lessons

Fig. 3. TutorAI running on a phone. (a) The user types any topic. (b) The generated lesson plays segment by segment; the diagram parts named by the current segment are highlighted while its audio plays. (c) Saved lessons replay fully offline, with no network.

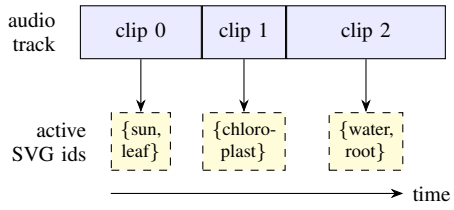


Fig. 4. Segment-level synchronization. Each audio clip's start and end define the window during which that segment's SVG element ids are highlighted, so the picture and the voice stay aligned with no word-level timing and no network.

A. Generation Flow on the Device

The topic screen takes the typed topic and calls the repository, which performs the asynchronous job dance: it posts the topic, then polls the job until the lesson is ready, surfacing progress to the view model as a state flow. The screen renders that state. When the lesson is ready the user can play it and, optionally, save it for offline use.

B. Synchronized Playback

The player is where the product idea becomes visible. The SVG is rendered inside a WebView as inline markup, with two small JavaScript functions to highlight and to clear a set of part names. Highlighting is a CSS effect, a glow and a slight scale, applied to the elements whose identifiers match. The audio is played with ExoPlayer as a playlist, one media item per segment. The index of the currently playing clip drives the highlight: when the player advances to clip k, the WebView highlights the part names of segment k. Because each clip's start and end define the highlight window, synchronization is exact and needs no word-level timing. Figure 4 shows this relationship. The transport controls map onto a small state machine with states for idle, loading, ready, playing, paused, seeking, and completed. Seeking simply stops the current clip, clears the highlight, and starts the chosen segment.

C. Offline and History

Saving a lesson downloads the SVG and every audio clip into app-private files and writes the lesson and its segments into Room. The player is deliberately source-agnostic: it loads its assets through a lambda, so a saved lesson loads from local file paths instead of network URLs and replays with no network at all. The history screen reads from Room, newest first, and supports deletion, which removes both the database rows and the files. Because there are no accounts and history is purely local, the privacy goal is met by construction.

TABLE I
DESIGN PATTERNS AND WHY EACH IS USED

Pattern	Purpose in TutorAI
Strategy	`LLMProvider` / `TTSProvider` interfaces; swap vendor without touching the graph
Abstract Factory	build a matching provider set from configuration
Adapter	wrap vendor SDKs behind our own interfaces
Builder	assemble a valid `Lesson` incrementally
Repository	hide the lesson cache and the asset store
Facade	one entry point over cache-check, graph, and persist
Pipeline + repair	staged generation with bounded loops (LangGraph)

VI. Engineering for Reliability and Reproducibility

A research prototype that only runs on its author's laptop is hard to trust or extend. We therefore put real effort into the engineering around the system.

A. Design Patterns

The code uses a small set of well-known patterns [32], each for a concrete reason, summarized in Table I. The patterns are not used for their own sake; they are the seams that make the system testable and swappable. The provider interfaces, for instance, are the single point at which the mock and Gemini implementations differ, which is exactly what lets the test suite stay key-free while the same code runs against live models in production.

B. Hermetic, Key-Free Testing

The backend test suite contains forty-eight tests that cover the validator, the provider factory, the filesystem repository, the graph end to end including both the repair-recovery and the exhausted-retries paths, the lessons API, audio handling, and

the Gemini adapters. The Gemini adapters are tested against a faked SDK client, so the suite never calls the live API and never needs a key. A session fixture disables environment loading and strips the application's environment variables, which makes the suite hermetic: it produces the same result on any machine and in continuous integration, with zero external calls. This is what lets the agent design be exercised cheaply and often. The Android side uses the repository interfaces as test seams in the same spirit.

C. Containerization and Continuous Delivery

The backend ships as a container with a health endpoint. Continuous integration runs the test suite and a container build sanity check on every change to the backend, and builds a debug application package on every change to the client. The delivery pipeline builds the backend image, pushes it to a container registry, and deploys it to a cloud host with a reverse proxy that obtains and renews a TLS certificate automatically, so the client always talks to the server over HTTPS with no cleartext exception. A separate workflow builds a signed release application package when a version tag is pushed and attaches it to a release. Path filters mean backend changes only trigger backend jobs and client changes only trigger client jobs. Together these give a reproducible path from a commit to a running, secured service and an installable application.

VII. Implementation Status and Verification

This is an implementation paper, so we report what the running system produces and what we verified, and we are explicit about what we have not yet measured.

End-to-end generation. With the real Gemini providers configured, the pipeline produced a complete lesson for the topic *Photosynthesis*, shown in Fig. 5: six narration segments totaling about forty-three seconds of audio, a well-formed SVG whose parts carry semantic names such as `sun`, `leaf`, and `chloroplast`, narration whose references all passed the deterministic validator, and real spoken audio for each segment. This exercises every node of the graph against live models. The saved library in Fig. 3(c) holds further lessons generated for unrelated topics—the Pythagorean theorem and Transformer architecture—so the pipeline is not tuned to a single subject.

The gate works in both directions. The graph tests confirm both that a lesson with a deliberately broken part reference is rejected and routed into the repair loop, and that when repair cannot fix the problem within the retry budget the job fails cleanly instead of caching a broken lesson. This is the behavior the design promises.

Reproducibility. The full suite of forty-eight backend tests passes with zero live API calls. The container builds and serves its health endpoint, and a mock-provider generation request returns the expected asynchronous response with no key present. The client builds an installable package, and the player and offline paths build against the Room and WebView integrations.

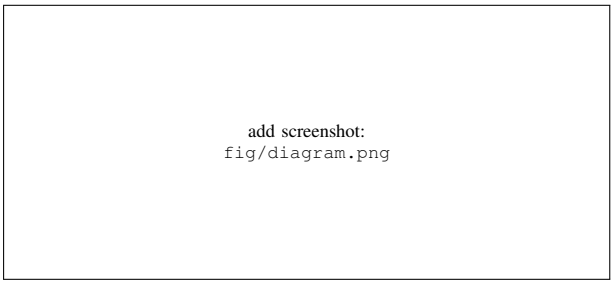


Fig. 5. The diagram generated for *Photosynthesis*, as shown in the app's full-screen view. Each labelled part—the sun, the leaf, the chloroplast, the water and carbon-dioxide inputs, and the glucose and oxygen outputs—is a separate SVG element with a stable id, which is exactly what lets the narration highlight it during playback.

TABLE II
Realized System at a Glance

Aspect	Realization
Backend	FastAPI, ≈2.0k LoC Python
Client	Android/Compose, ≈3.8k LoC Kotlin
Generation	LangGraph: plan → draw → critique/ refine → narrate → validate → repair
Gate	deterministic non-LLM validator
Models	text + speech via swappable adapters
Caching	stateless, topic-keyed, coalesced jobs
Offline	Room + file store; network-free replay
Tests	48 hermetic, key-free backend tests
Delivery	container + CI/CD + auto-HTTPS

What we have not measured. We have not yet run a formal user study on learning outcomes, nor a large quantitative study of diagram quality across many topics, nor on-device latency and battery measurements across a fleet of phones. We report the system and its verified behavior, and we flag these studies as future work rather than claiming results we did not collect. Table II summarizes the realized system.

VIII. Discussion and Limitations

Several limitations are worth naming plainly.

Diagram quality has a ceiling. A language model drawing SVG is good at clear, well-known concepts and weaker at dense or unusual ones. The critique-and-refine loop raises the floor but does not guarantee a beautiful result. The deterministic gate guarantees *consistency*, not aesthetic quality; a valid lesson can still be a plain one.

Synchronization is at the segment level. We light up a group of parts for a whole sentence, not each part as its word is spoken. This was a deliberate choice for determinism and offline play. Word-level highlighting would be more precise but would reintroduce timing data and fragility.

Cost and latency for cold topics. The first request for a new topic pays for several model calls and the speech synthesis, and takes seconds. The shared cache hides this for repeated topics, and bounded loops cap the worst case, but the cold path is inherently the slow path.

Security of generated markup. Rendering model-produced SVG in a WebView means the markup must be treated as untrusted. We restrict the WebView to our own highlight script and avoid executing content-supplied script, but this is an area that deserves ongoing care.

Evaluation breadth. As noted, our claims are about the system's behavior, not about measured learning gains. A controlled study comparing TutorAI lessons against static diagrams and against plain text is the most important next step.

IX. Conclusion and Future Work

We presented TutorAI, a system that turns a typed topic into a short, self-explaining lesson: a named vector diagram, segment-level narration tied to those names, and per-segment audio that drives synchronized highlighting on the device, fully offline. The core insight is architectural rather than about any single model. By splitting generation into a small graph of steps, adding a bounded critique-and-refine loop for the drawing and a bounded repair loop for the narration, and putting a deterministic, non-LLM validator in front of the cache, we make a probabilistic generator reliable enough to power a feature that depends on exact part-to-word correspondence. Around this core, a stateless caching backend keeps cost down, swappable adapters keep the vendor choice soft, and an offline-first client keeps the lesson usable without a network. Hermetic tests, containerization, and continuous delivery make the whole thing reproducible.

Future work falls into three groups. First, evaluation: a controlled learning study, a larger automatic study of diagram quality across many topics, and on-device latency and energy measurements. Second, capability: optional word-level highlighting where timing allows, richer diagram styles, a research or planning step in front of the drawing for harder topics, and multi-language lessons. Third, scale: promoting the in-process worker and filesystem store to a separate worker, a database, and a cloud object store behind the same interfaces. None of these change the central design; they extend it.

Acknowledgment

The authors used a large language model–based AI assistant (Anthropic's Claude) to assist in drafting and editing the text of this manuscript. The system described in this paper—its conception, design, implementation, and the verification reported—is the authors' own work. The authors reviewed all text and take full responsibility for the content of this paper.

References

[1] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[2] LangChain, "LangGraph: Building stateful, multi-actor applications with LLMs," software documentation. [Online]. Available: https://langchain-ai.github.io/langgraph/

[3] B. S. Bloom, "The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.

[4] J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, "Cognitive tutors: Lessons learned," *The Journal of the Learning Sciences*, vol. 4, no. 2, pp. 167–207, 1995.

[5] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.

[6] E. Kasneci *et al.*, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, 102274, 2023.

[7] A. Paivio, "Dual coding theory: Retrospect and current status," *Canadian Journal of Psychology*, vol. 45, no. 3, pp. 255–287, 1991.

[8] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[9] R. E. Mayer, Ed., *The Cambridge Handbook of Multimedia Learning*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2014.

[10] R. E. Mayer and R. Moreno, "Nine ways to reduce cognitive load in multimedia learning," *Educational Psychologist*, vol. 38, no. 1, pp. 43–52, 2003.

[11] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[12] T. B. Brown *et al.*, "Language models are few-shot learners," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.

[13] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[14] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[15] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[16] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[17] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[18] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," arXiv:2304.05128, 2023.

[19] H. Chase, "LangChain," open-source software, 2022. [Online]. Available: https://github.com/langchain-ai/langchain

[20] P. Liu *et al.*, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[21] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10684–10695.

[22] World Wide Web Consortium (W3C), "Scalable Vector Graphics (SVG) 1.1 (Second Edition)," W3C Recommendation, 2011. [Online]. Available: https://www.w3.org/TR/SVG11/

[23] A. Carlier, M. Danelljan, A. Alahi, and R. Timofte, "DeepSVG: A hierarchical generative network for vector graphics animation," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[24] J. A. Rodriguez *et al.*, "StarVector: Generating scalable vector graphics code from images," arXiv:2312.11556, 2023.

[25] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[26] A. van den Oord *et al.*, "WaveNet: A generative model for raw audio," arXiv:1609.03499, 2016.

[27] J. Shen *et al.*, "Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 4779–4783.

[28] A. Feyerke, "Designing offline-first web apps," *A List Apart*, no. 373, 2013. [Online]. Available: https://alistapart.com/article/offline-first/

[29] Google, "Guide to app architecture," Android Developers documentation. [Online]. Available: https://developer.android.com/topic/architecture

[30] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.

[31] A. Wiggins, "The Twelve-Factor App," 2017. [Online]. Available: https://12factor.net/

[32] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.